O Problema de Visita de Polígonos*

Gabriel F. Ushijima† Ernesto G. Birgin†

5 de março de 2026

Resumo

Este trabalho apresenta diferentes implementações de algoritmos para o Problema de Visita de Polígonos (*Touring Polygons Problem* - TPP), que consiste em determinar um caminho de comprimento mínimo na norma Euclidiana entre um ponto inicial e um ponto final, visitando uma sequência ordenada de polígonos no plano. Em sua formulação geral, o problema é NP-difícil. Por esse motivo, inicialmente considera-se uma versão simplificada na qual os polígonos são convexos e disjuntos, para a qual se apresenta um algoritmo exato baseado em mapas de último passo. Posteriormente, para o caso não convexo, particiona-se cada polígono não convexo em um conjunto de polígonos convexos e aplica-se o algoritmo desenvolvido para o caso convexo a cada combinação possível de polígonos resultantes da partição. Dado que essa solução apresenta limitações de eficiência, discutem-se estratégias de otimização inteira mista para melhorar o desempenho dessa abordagem. Por fim, realizam-se experimentos computacionais para avaliar o desempenho dos algoritmos propostos em instâncias de diferentes tamanhos e características, comparando-os com *solvers* comerciais. Os resultados demonstram a eficácia das abordagens desenvolvidas em uma ampla variedade de cenários.

Palavras-chave: problema de visita de polígonos, otimização geométrica, complexidade computacional, experimentos numéricos.

1 Introdução

O Problema de Visita de Polígonos (*Touring Polygons Problem* - TPP), introduzido em [2], constitui um problema de otimização geométrica que consiste em determinar um caminho de comprimento mínimo na norma Euclidiana que parte de um ponto inicial, visita uma sequência ordenada de polígonos e alcança um ponto final no plano. Esse tipo de problema surge naturalmente em aplicações de roteirização e planejamento de trajetórias, especialmente no contexto de veículos autônomos. Nesses cenários, torna-se necessário garantir a visita eficiente a regiões específicas do espaço, como em tarefas de inspeção automatizada de armazéns [3, 4]. O problema pode ser interpretado como um caso específico do Problema do Caixeiro Viajante com Regiões (*Traveling Salesman Problem with Neighborhoods* - TSPN) [1]. No TSPN, o objetivo consiste em encontrar um caminho de comprimento mínimo que visite um conjunto de regiões arbitrárias no plano; no TPP, especificamente, essas regiões são representadas por polígonos e a ordem de visita é pré-determinada.

Dentre as diversas variações do problema, considera-se inicialmente uma versão simplificada na qual os polígonos são convexos e disjuntos. Considera-se ainda que a distância é medida na norma Euclidiana. Para esse caso, apresentam-se três abordagens distintas para resolver o problema de forma exata. As duas primeiras baseiam-se em ideias descritas em [2], enquanto a terceira constitui

*Esse trabalho foi parcialmente apoiado pela agência FAPESP (processo 2025/13861-1).

†Instituto de Matemática e Estatística, Universidade de São Paulo, Rua do Matão, 1010, Cidade Universitária, 05508-090, São Paulo, SP, Brazil (e-mail: gabriel_ushijima@ime.usp.br, egbirgin@ime.usp.br).

uma extensão que utiliza técnicas de memoização para melhorar o desempenho em diversos casos explorados nas seções subsequentes.

A primeira abordagem consiste em uma implementação direta das ideias descritas em [2], baseada em mapas de último passo, que resolve o problema em tempo $O(n^2)$, onde n representa a soma do número de vértices dos k polígonos de entrada. Essa implementação tem como objetivo familiarizar o leitor com os conceitos e técnicas fundamentais para a resolução do TPP, tais como as regiões de primeiro contato e os mapas de último passo, que serão utilizados em todas as implementações subsequentes.

A segunda abordagem aprimora a eficiência da primeira ao empregar uma estratégia de busca binária para a localização de pontos nos mapas de último passo, resolvendo o problema em tempo $O(nk \log(n/k))$, conforme descrito em [2].

A terceira abordagem utiliza uma técnica de memoização com o objetivo de evitar cálculos redundantes e reduzir o custo computacional. Embora a complexidade de pior caso permaneça $O(nk \log(n/k))$, essa abordagem pode apresentar desempenho significativamente superior na prática, dependendo da distribuição de vértices entre os polígonos. Essa estratégia é, ao que tudo indica, original, não tendo sido descrita no artigo de referência nem em outros trabalhos consultados.

Em seguida, aborda-se o caso geral do TPP, no qual os polígonos podem ser não convexos. Para isso, particiona-se cada polígono não convexo em um conjunto de polígonos convexos e aplicam-se os algoritmos desenvolvidos para o caso convexo a cada combinação possível de polígonos convexos resultantes da partição. Essa abordagem apresenta, *a priori*, limitações de eficiência; assim, busca-se otimizar o processo por meio de técnicas de otimização combinatória.

Alternativamente, o caso geral do TPP pode ser resolvido modelando-o como um problema de programação inteira mista (*Mixed Integer Linear Programming* - MILP) e utilizando *solvers* comerciais, como o Gurobi, para obter uma solução. Essa abordagem também resolve o problema de forma exata. Dessa forma, demonstra-se que a implementação proposta é competitiva mesmo quando comparada a *solvers* comerciais de alto desempenho.

O restante deste trabalho está organizado da seguinte forma. As três abordagens para resolver o TPP no caso convexo são apresentadas na Seção 2, juntamente com a análise de complexidade e experimentos numéricos. A Seção 3 aborda o caso geral do TPP com polígonos não convexos, apresentando a estratégia de particionamento e otimização combinatória, bem como a modelagem do problema como um MILP. Conclusões e perspectivas para trabalhos futuros são discutidas na seção final.

2 O Problema de Visita de Polígonos Convexos

Inicialmente, considera-se o TPP em sua versão mais simples, na qual os polígonos são convexos e disjuntos, denominada TPP Irrestrito. Utilizam-se tanto a formulação quanto a solução apresentadas em [2] como base para as implementações desenvolvidas. Para os propósitos deste trabalho, define-se o problema da seguinte forma:

> **TPP Convexo:** Dados pontos $s, t \in \mathbb{R}^2$ e polígonos convexos disjuntos $P_1, \ldots, P_k$, determinar pontos $p_i \in P_i$ para $i = 1, 2, \ldots, k$ de forma que o caminho que parte de s, passa por $p_1, \ldots, p_k$ e termina em t possua comprimento mínimo.

Todas as implementações subsequentes fundamentam-se nas definições e propriedades apresentadas em [2]. A seguir, apresentam-se as principais definições que serão utilizadas. Deste ponto em diante, supõe-se que um polígono é representado por uma sequência de vértices em sentido anti-horário. Supõe-se ainda que $P_0 = \{s\}$ e $P_{k+1} = \{t\}$.

Caminho Parcial Ótimo (i-path): Um i-path ($0 \leq i \leq k$) até um ponto $p \in \mathbb{R}^2$ é um caminho de comprimento mínimo que se inicia em s, termina em p e visita os polígonos $P_1, P_2, \ldots, P_i$

nessa ordem. Considerando-se a distância Euclidiana, todo i-path é formado pela união de segmentos de reta; assim, tratam-se caminhos como sequências ordenadas de pontos em $\mathbb{R}^2$. Além disso, uma vez que se trabalha com polígonos convexos, todo i-path é único, o que se tornará evidente a partir da definição de mapa de último passo e de suas propriedades, apresentadas adiante. Por esse motivo, referir-se-á a um i-path como **o** i-path até um ponto p.

Cabe observar que o 0-path até p consiste simplesmente no segmento de reta $\overline{sp}$ e o k-path até t corresponde exatamente à solução do problema.

A Figura 1 ilustra um exemplo de 2-path até t. Note-se que a ordem deve ser respeitada, pois visitar P_2 antes de P_1 seria mais eficiente neste caso.

Região de Primeiro Contato (T_i): Denota-se por T_i ($1 \leq i \leq k$) a região de primeiro contato de um polígono P_i. Seja $p \in P_i$ e seja q o ponto anterior a p no $(i-1)$-path até p. Tem-se que $p \in T_i$ se e somente se $\overline{pq} \cap P_i = p$. Em outras palavras, T_i consiste nos pontos p pertencentes ao perímetro de P_i tais que o último segmento do $(i-1)$-path até p não atravessa o interior de P_i. É imediato verificar que apenas pontos na fronteira de P_i pertencem a T_i. Observa-se ainda que, como P_i é convexo, T_i é sempre composta por uma região contínua de arestas de P_i.

A Figura 2 ilustra um exemplo das regiões de primeiro contato T_1 e T_2 dos polígonos P_1 e P_2, respectivamente.

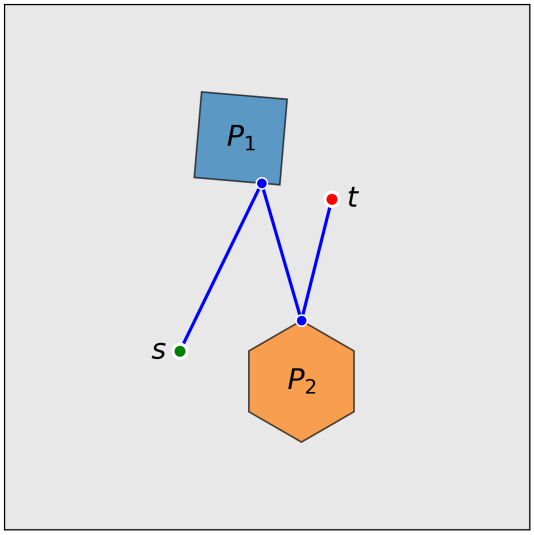


Figura 1: Caminho mínimo de s até t visitando P_1 e P_2.

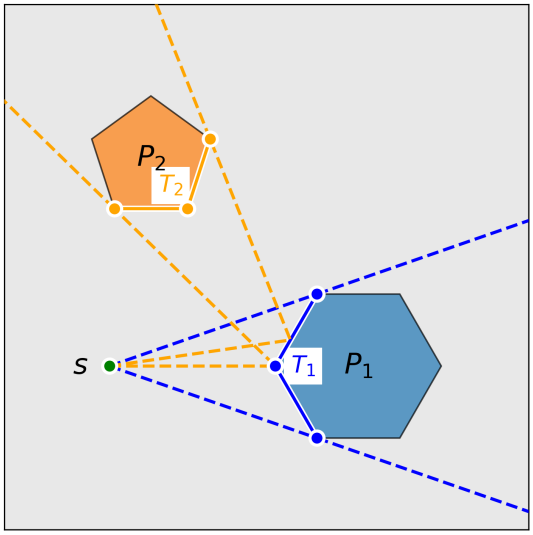


Figura 2: Região de primeiro contato T_1 de P_1 e T_2 de P_2.

Mapa de Último Passo (S_i): Denota-se por S_i ($1 \leq i \leq k$) o mapa de último passo de um polígono P_i, que consiste em uma partição do plano em um número finito de regiões. Na partição podem existir duas ou mais regiões do tipo "região de vértice", uma ou mais regiões do tipo "região de aresta" e exatamente uma região do tipo "região de atravessar". Cada região de vértice em S_i está associada a um vértice v de P_i, assim como cada região de aresta está associada a uma aresta e de P_i. Apenas vértices e arestas em T_i possuem uma região associada em S_i.

Um ponto $p \in \mathbb{R}^2$ pertence a uma região de vértice associada ao vértice v de P_i se e somente se o i-path até p possui $\overline{vp}$ como último segmento. Analogamente, um ponto $p \in \mathbb{R}^2$ pertence a uma região de aresta associada à aresta e de P_i se e somente se o último segmento do i-path até p inicia-se no interior de e. Por último, um ponto $p \in \mathbb{R}^2$ pertence à região de atravessar de P_i se e somente se não pertence a nenhuma região de vértice ou de aresta.

Cabe mencionar que, a partir desta definição, segue que os pontos na fronteira da região de vértice pertencem a ela, os pontos no interior de uma aresta de P_i em T_i estão na região dessa aresta e os pontos no interior de P_i e nas arestas de P_i que não estão em T_i pertencem à região de atravessar. Dessa forma, S_i configura uma partição do plano, isto é, cada ponto do plano pertence a exatamente uma região do mapa de último passo S_i de P_i.

A Figura 3 apresenta um exemplo de mapa de último passo de um polígono P_1. As regiões de vértice são representadas em vermelho, as regiões de aresta em verde e a região de atravessar em azul. Para cada região, inclui-se um exemplo de ponto pertencente a ela, bem como o 1-path até esse ponto, ilustrando a definição de cada tipo de região.

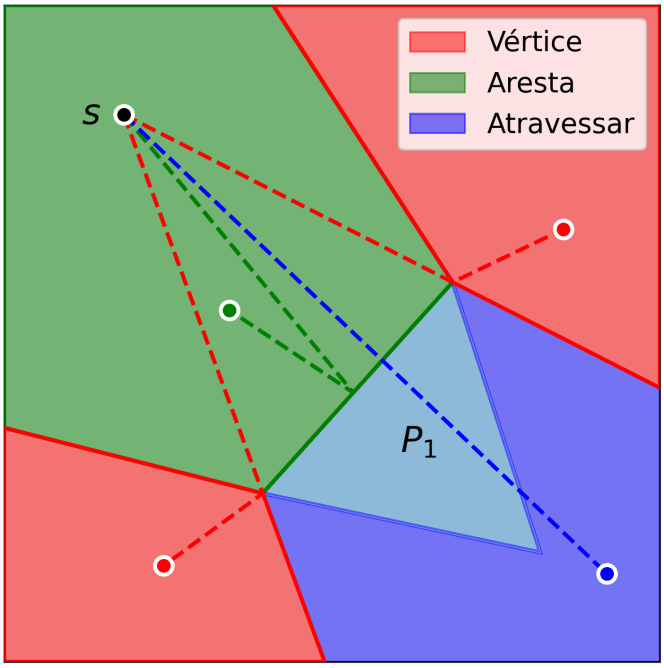


Figura 3: Exemplo do mapa de último passo S_1 de P_1. Os pontos em vermelho pertencem a regiões de vértice, os pontos em verde pertencem a regiões de aresta e os pontos em azul pertencem à região de atravessar. Inclui-se o 1-path até cada um deles na figura.

Sejam $p \in \mathbb{R}^2$ e $1 \leq i \leq k$. Dispondo-se de $S_1, \ldots, S_i$, pode-se determinar o i-path até p de forma eficiente, dependendo do tipo de região à qual p pertence no mapa de último passo S_i, conforme descrito a seguir:

Região de Vértice: Se p pertence a uma região associada a um vértice v de P_i, então o último segmento do i-path até p é o segmento $\overline{vp}$. Assim, o i-path até p é simplesmente o $(i-1)$-path até v, seguido do segmento $\overline{vp}$.

A Figura 4 ilustra essa situação. Tem-se o ponto p^1 precedido por q^1 e p^2 precedido por q^2, sendo que tanto q^1 quanto q^2 são vértices de P_2. Optou-se por incluir tanto p^1 quanto p^2 para ilustrar que o mapa de último passo S_2 fornece informações apenas sobre o **último segmento** do 2-path, não sobre o caminho completo, ou seja, não é possível afirmar nada a respeito do último segmento do caminho até q^1 ou q^2. De fato, no exemplo escolhido, observa-se que o último segmento até q^1 parte de um vértice de P_1, enquanto o último segmento até q^2 parte do interior de uma aresta de P_1. Para determinar o caminho completo até p^1 ou p^2, é necessário consultar S_1 além de S_2.

Região de Atravessar: Se p pertence à região de atravessar de P_i, então o último segmento do i-path até p atravessa o interior de P_i. Nesse caso, o i-path até p é dado pelo $(i-1)$-path

até p. A Figura 5 ilustra essa situação, na qual o 2-path até p coincide com o 1-path até p e atravessa P_2.

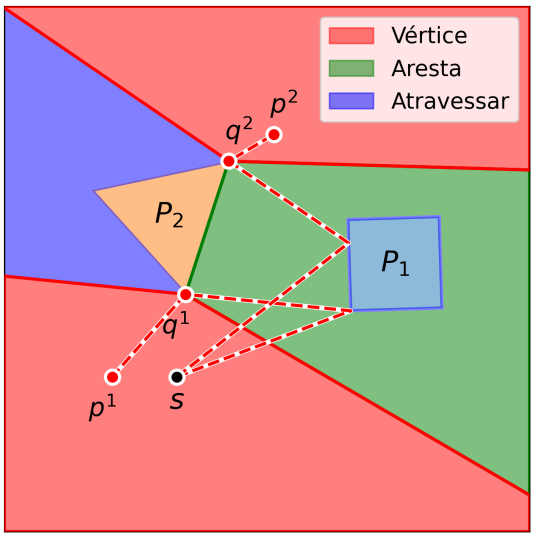


Figura 4: 2-path até p^1 e p^2, que estão em regiões de vértice.

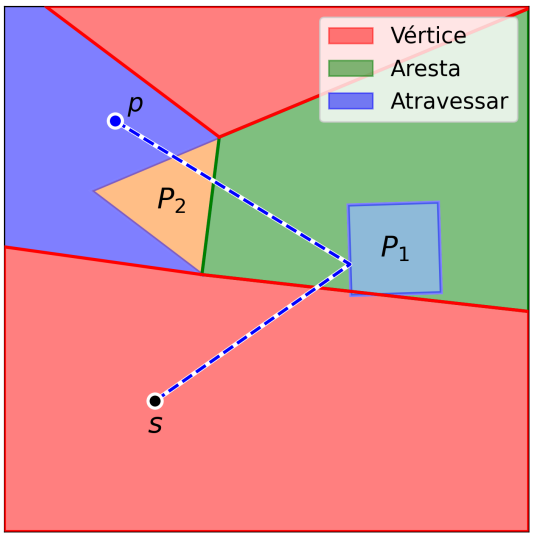


Figura 5: 2-path até p, que está em uma região de atravessar.

Região de Aresta: Se p pertence a uma região associada a uma aresta $e = \overline{uv}$ de P_i, então reflete-se o ponto p em relação à reta que contém e, obtendo-se um ponto refletido p'. Calcula-se o $(i-1)$-path até p', cujo último segmento parte de um ponto q', e denota-se por q a interseção do segmento $\overline{q'p'}$ com a aresta e. O i-path até p é então formado pela parte do $(i-1)$-path até p' que se estende desde o início (ponto s) até o ponto q, seguida do segmento $\overline{qp}$. A Figura 6 ilustra essa situação, mostrando como o ponto deve ser refletido e como se utiliza o $(i-1)$-path até p' para determinar o i-path até p.

A seguir, apresentamos as fórmulas para o cálculo do ponto refletido p' e do ponto de interseção q. Utilizamos o operador $\cdot$ para denotar o produto escalar, o operador $\|\cdot\|$ para denotar a norma euclidiana e o operador $\times$ para denotar o produto vetorial[1], todos definidos em $\mathbb{R}^2$:

$$p' = -p + 2u + 2\frac{(p-u)\cdot(v-u)}{\|v-u\|^2}(v-u), \tag{1}$$

$$q = q' + \frac{(u-q')\times(v-u)}{(p'-q')\times(v-u)}(p'-q'). \tag{2}$$

Com base nessas construções, implementamos o Algoritmo 1, que, dados um ponto $p \in \mathbb{R}^2$ e um índice $1 \leq i \leq k$, retorna o i-path até p como uma sequência de pontos em $\mathbb{R}^2$, desconsiderando o último ponto do caminho, p, que é conhecido. Assumimos também a existência de uma função `LocalizaPonto`, que, dados $p \in \mathbb{R}^2$ e $1 \leq i \leq k$, localiza o ponto p no mapa de último passo S_i de P_i e retorna a região R à qual p pertence. Essa região carrega tanto o tipo de região à qual o ponto pertence quanto o vértice v da região, se for uma região de vértice, ou a aresta e, se for uma região de aresta.

O Algoritmo 1 permite calcular o i-path até qualquer ponto $p \in \mathbb{R}^2$ de forma eficiente, podendo ser utilizado para obter o k-path até t, que corresponde exatamente à solução do problema. O algoritmo recebe como entrada o ponto p, o ponto inicial s, o índice i, os polígonos P_ℓ para $\ell =$

[1]Para dois vetores $a = (a_x, a_y)$ e $b = (b_x, b_y)$, temos $a \times b = a_x b_y - a_y b_x$.

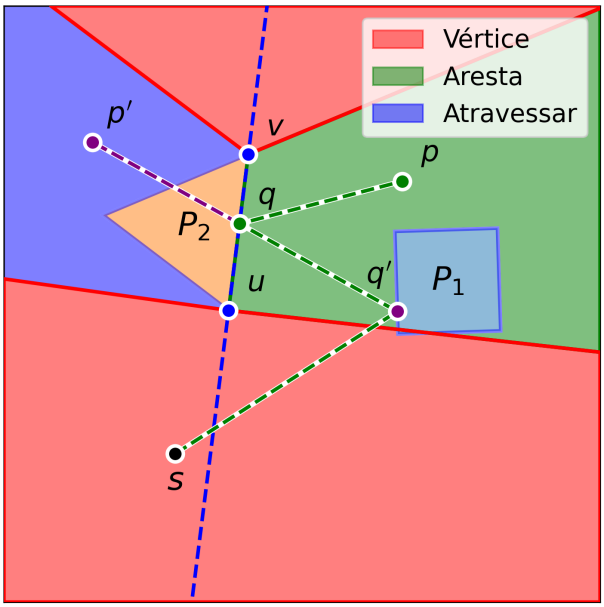


Figura 6: 2-path até p, que pertence a uma região de aresta associada à aresta $\overline{uv}$ de P_2. Ilustramos o ponto refletido p', o ponto que antecede p', q', e o ponto de interseção q.

Algoritmo 1: Consulta Caminho

Entrada: $p, i, s, (P_1, \ldots, P_i), (T_1, \ldots, T_i), (S_1, \ldots, S_i)$
Saída : i-path até p como uma sequência de pontos em $\mathbb{R}^2$, desconsiderando p.

define ConsultaCaminho$(p, i, s, (P_1, \ldots, P_i), (T_1, \ldots, T_i), (S_1, \ldots, S_i))$**:**
  se $i = 0$ **:**
    retorna $[s]$
  $R \leftarrow$ LocalizaPonto(p, P_i, T_i, S_i)
  se R corresponde a um vértice v de P_i **:**
    $\mathbb{P} \leftarrow$ ConsultaCaminho$(v, i-1, s, (P_1, \ldots, P_{i-1}), (T_1, \ldots, T_{i-1}), (S_1, \ldots, S_{i-1}))$
    retorna $\mathbb{P} \oplus [v]$ `// Adiciona` v `ao final do caminho` $\mathbb{P}$
  senão se R corresponde a uma aresta $e = \overline{uv}$ de P_i **:**
    $p' \leftarrow$ reflexão de p em relação à reta que contém e (Veja 1)
    $\mathbb{P} \leftarrow$ ConsultaCaminho$(p', i-1, s, (P_1, \ldots, P_{i-1}), (T_1, \ldots, T_{i-1}), (S_1, \ldots, S_{i-1}))$
    $q' \leftarrow$ último ponto de $\mathbb{P}$
    $q \leftarrow$ ponto de interseção entre as retas que passam por $\overline{q'p'}$ e e (Veja 2)
    retorna $\mathbb{P} \oplus [q]$ `// Adiciona` q `ao final do caminho` $\mathbb{P}$
  senão se R corresponde a atravessar P_i **:**
    retorna ConsultaCaminho$(p, i-1, s, (P_1, \ldots, P_{i-1}), (T_1, \ldots, T_{i-1}), (S_1, \ldots, S_{i-1}))$

$1, \ldots, i$, e suas respectivas regiões de primeiro contato T_i e mapas de último passo S_i, e devolve o caminho. O algoritmo é recursivo e faz uso exclusivamente da função `LocalizaPonto`, que definiremos em breve.

Frequentemente, interessa-nos apenas o último segmento do i-path até p. Ainda, no caso em que um ponto pertence a uma região associada a um vértice de P_i, é possível determinar o último segmento diretamente, sem a necessidade de calcular o restante do caminho. Dessa forma, implementamos o Algoritmo 2, que, dados um ponto $p \in \mathbb{R}^2$ e um índice $1 \leq i \leq k$, retorna o ponto inicial do último segmento do i-path até p. O algoritmo recebe como entrada os mesmos

dados do Algoritmo 1.

Algoritmo 2: Consulta Último Passo

Entrada: $p, i, s, (P_1, \ldots, P_i), (T_1, \ldots, T_i), (S_1, \ldots, S_i)$
Saída : Ponto que precede p no i-path até p.

```
1  define ConsultaÚltimoPasso(p, i, s, (P_1, ..., P_i), (T_1, ..., T_i), (S_1, ..., S_i)):
2      se i = 0 :
3          retorna s
4      R ← LocalizaPonto(p, P_i, T_i, S_i)
5      se R corresponde a um vértice v de P_i :
6          retorna v
7      senão se R corresponde a uma aresta e = uv de P_i :
8          p' ← reflexão de p em relação à reta que contém e (Veja 1)
9          q' ← ConsultaÚltimoPasso(p', i − 1, s, (P_1, ..., P_{i−1}), (T_1, ..., T_{i−1}), (S_1, ..., S_{i−1}))
10         q ← ponto de interseção entre q'p' e e (Veja 2)
11         retorna q
12     senão se R corresponde a atravessar P_i :
13         retorna
             ConsultaÚltimoPasso(p, i − 1, s, (P_1, ..., P_{i−1}), (T_1, ..., T_{i−1}), (S_1, ..., S_{i−1}))
```

Esses dois algoritmos serão utilizados em todas as implementações discutidas a seguir. A distinção entre elas residirá na implementação do algoritmo `LocalizaPonto` e na forma como calculamos as regiões de primeiro contato T_i e os mapas de último passo S_i para cada polígono P_i.

2.1 Primeira Abordagem

Em alto nível, o algoritmo proposto para resolver o TPP Convexo opera da seguinte maneira: para cada $1 \leq i \leq k$, calcula-se a região de primeiro contato T_i de P_i utilizando $T_1, \ldots, T_{i-1}$ e $S_1, \ldots, S_{i-1}$ por meio do Algoritmo 3; em seguida, computa-se o mapa de último passo S_i de P_i empregando $T_1, \ldots, T_i$ e $S_1, \ldots, S_{i-1}$ através do Algoritmo 4. Ao final, determina-se o k-path até t mediante o Algoritmo 1. A seguir, detalham-se as implementações de cada uma dessas etapas.

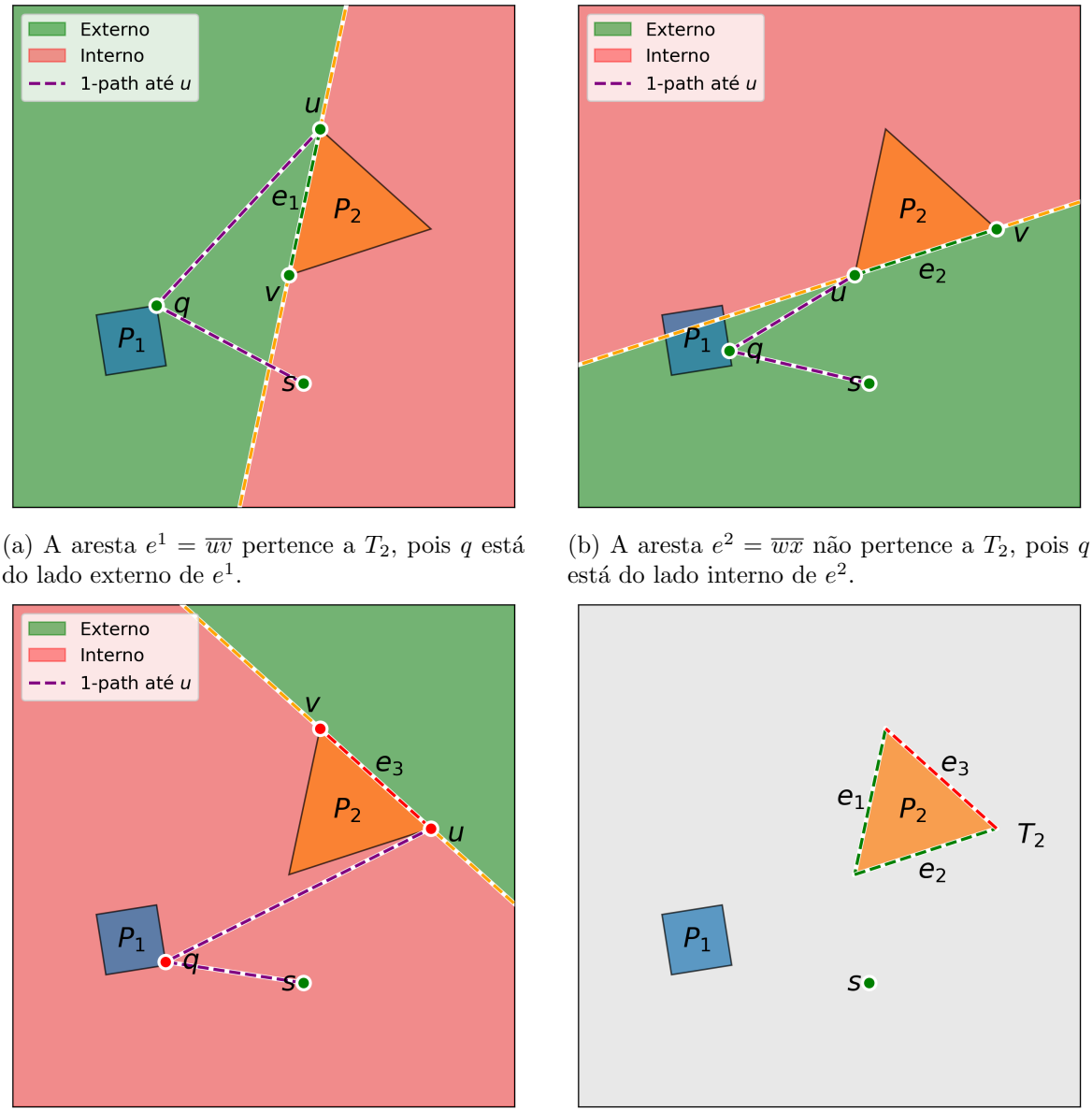
2.1.1 Região de Primeiro Contato

Primeiramente, objetiva-se determinar a região de primeiro contato T_i de um polígono P_i. Ressalta-se que conhecer apenas quais **vértices** de P_i pertencem a T_i não é suficiente para determinar T_i, uma vez que T_i pode conter todos os vértices de P_i e, ainda assim, possuir uma aresta que não pertence a T_i, sobre a qual não se dispõe de informação. Por esse motivo, concentra-se em determinar quais **arestas** de P_i pertencem a T_i. Considerando-se a definição de região de primeiro contato, verifica-se que vale a seguinte propriedade:

> **Lemma 2.1** *Seja $e = \overline{uv}$ uma aresta de P_i $(1 \leq i \leq k)$, p um ponto qualquer dessa aresta (incluindo as extremidades) e q o ponto que precede p no $(i-1)$-path até p. Tem-se que $e \in T_i$ se, e somente se, q está do lado externo de e, isto é, $(v-u)\times(q-u) < 0$. Caso contrário, $e \notin T_i$.*

Assim, para determinar T_i, basta realizar o teste descrito acima para cada aresta de P_i. Essa abordagem é ilustrada na Figura 7, na qual, para cada aresta $e = \overline{uv}$ de P_2, calcula-se o ponto q que precede u no 1-path até u e verifica-se se ele se encontra do lado externo de e. Representam-se em verde as arestas que pertencem a T_2 e em vermelho aquelas que não pertencem. Essa mesma

ideia é implementada no Algoritmo 3, o qual recebe como entrada o índice i, o ponto inicial s, os polígonos $P_1, \ldots, P_i$ e suas respectivas regiões de primeiro contato $T_1, \ldots, T_{i-1}$ e mapas de último passo $S_1, \ldots, S_{i-1}$, e retorna a região de primeiro contato T_i de P_i como um conjunto de arestas de P_i.



(a) A aresta $e^1 = \overline{uv}$ pertence a T_2, pois q está do lado externo de e^1.

(b) A aresta $e^2 = \overline{wx}$ não pertence a T_2, pois q está do lado interno de e^2.

(c) A aresta $e^3 = \overline{yz}$ não pertence a T_2, pois q está do lado interno de e^3.

(d) Resultado final: apenas as arestas e^1 e e^2 pertencem a T_2.

Figura 7: Utilização do Algoritmo 3 para determinar a região de primeiro contato de P_2, realizando o teste para cada aresta do polígono. Representam-se em verde as arestas que pertencem a T_2 e em vermelho aquelas que não pertencem.

2.1.2 Mapa de Último Passo

Uma vez determinada T_i, pode-se calcular o mapa de último passo S_i de P_i. Para tal, retomam-se primeiramente as condições de otimalidade local discutidas em [2, **Local Optimality Conditions.**], as quais permitem determinar o mapa de último passo S_i de um polígono P_i a partir da

Algoritmo 3: Região de Primeiro Contato

Entrada: $i, s, (P_1, \ldots, P_i), (T_1, \ldots, T_{i-1}), (S_1, \ldots, S_{i-1})$

Saída : Região de primeiro contato T_i de P_i representada como um conjunto de arestas de P_i.

```
1 define RegiãoDePrimeiroContato(i, s, (P_1, ..., P_i), (T_1, ..., T_{i-1}), (S_1, ..., S_{i-1)):
2     T_i ← ∅
3     para cada cada aresta e de P_i :
4         u ← vértice anterior de e
5         v ← vértice posterior de e
6         q ← ConsultaÚltimoPasso(u, i − 1, s, (P_1, ..., P_{i−1}), (T_1, ..., T_{i−1}), (S_1, ..., S_{i−1}))
7         se (v − u) × (q − u) < 0 :
8             T_i ← T_i ∪ e
9     retorna T_i
```

região de primeiro contato T_i:

> **Lemma 2.2** *Seja v um vértice de um polígono convexo qualquer P com arestas incidentes e^1 e e^2, e suponha-se que se deseja visitar P partindo de $q \in \mathbb{R}^2$ e terminando em $p \in \mathbb{R}^2$. Para que o caminho mínimo de q até p passe por v, é necessário que p esteja contido no cone definido por v e por um par ordenado de raios que partem de v, determinados a partir de e^1 e e^2 nessa ordem. O raio associado a uma aresta e incidente a v será $v - q$ se q está do lado interno de e, ou $q' - v$, onde q' é a reflexão de q em relação à reta perpendicular a e que passa por v, caso contrário.*

Com isso em mente, seja v um vértice de P_i com arestas incidentes e^1 e e^2, e seja q o ponto que precede v no $(i-1)$-path até v. Observe-se que, como v pertence a e^1 e e^2, tem-se que q está do lado externo de e^1 se, e somente se, $e^1 \in T_i$, e q está do lado externo de e^2 se, e somente se, $e^2 \in T_i$. Assim, é possível determinar os raios associados a e^1 e e^2 a partir de T_i.

Por definição, o $(i-1)$-path até v possui comprimento mínimo. Desse modo, se for determinado que alcançar p visitando P_i através de v é localmente ótimo, então pode-se afirmar que o i-path até p é dado pelo $(i-1)$-path até v seguido do segmento $\overline{vp}$. Dessa forma, conclui-se que os raios associados a v, calculados a partir de T_i, descrevem exatamente a região de vértice associada a v em S_i. Assim, iterando sobre cada vértice de P_i e calculando os raios associados a ele a partir de T_i, é possível determinar todas as regiões de vértice de S_i.

Essa ideia é implementada no Algoritmo 4, onde, para cada vértice v de P_i, calculam-se os raios associados a v e associa-se o par ordenado desses raios a v em S_i. Observe-se que, mesmo que um vértice v de P_i não possua nenhuma aresta incidente em T_i, ainda assim é possível determinar os raios associados a v, os quais serão coincidentes. Isso contradiz a definição formal de região de vértice, que define tais regiões apenas para vértices em T_i. Todavia, essa definição dos raios para vértices que não pertencem a T_i será útil posteriormente. Por ora, mantém-se em mente que o S_i em código contém mais informação do que o S_i definido formalmente.

Uma vez obtidas todas as regiões de vértice de S_i, pode-se afirmar que as regiões de aresta de S_i são exatamente as regiões entre os pares ordenados de raios associados a vértices adjacentes em P_i, e que a região de atravessar corresponde ao complemento dessas regiões. Assim, com as regiões de vértice determinadas, é possível estabelecer as regiões de aresta e a região de atravessar.

Algoritmo 4: Mapa de Último Passo

Entrada: $i, s, (P_1, \ldots, P_i), (T_1, \ldots, T_i), (S_1, \ldots, S_{i-1})$

Saída : Mapa de último passo S_i de P_i como uma associação de pares ordenados de raios aos vértices das arestas de T_i.

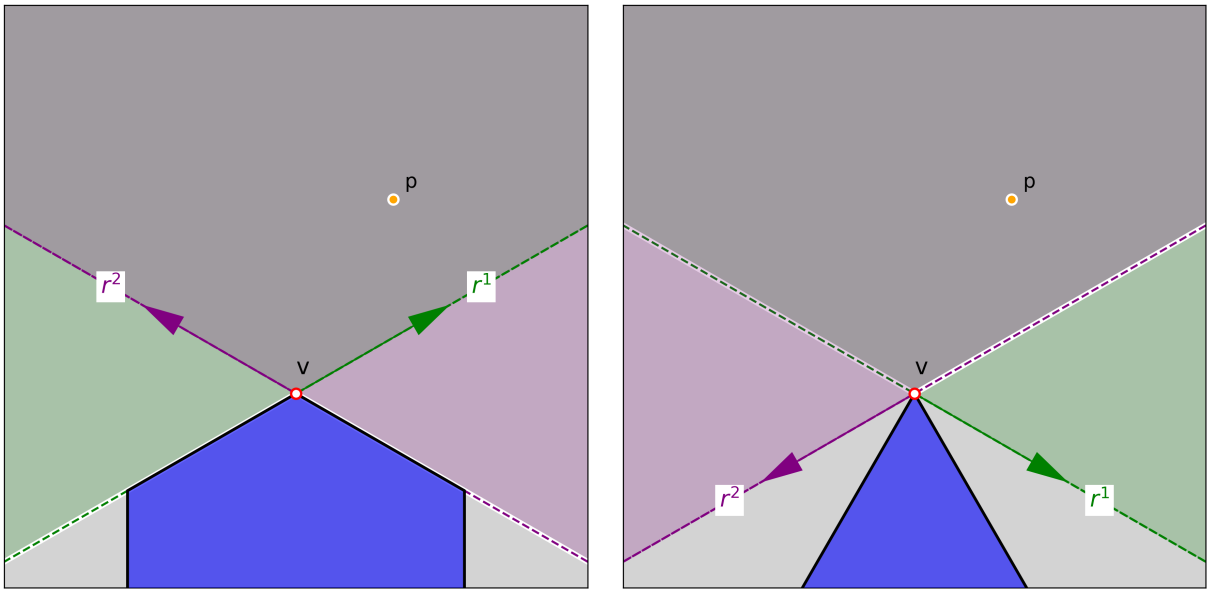
```
// Função auxiliar para calcular o raio refletido na aresta e incidente a v
   onde q é o ponto que precede v no (i − 1)-path até v.
define Reflexão(q, v, e):
    se e ∈ T_i :
        q' ← reflexão de q em relação à reta perpendicular a e que passa por v (Veja 1)
        retorna q' − v
    senão
        retorna v − q
define MapaDeÚltimoPasso(i, s, (P_1, ..., P_i), (T_1, ..., T_i), (S_1, ..., S_{i−1})):
    S_i ← ∅
    para cada vértice v de P_i :
        e^1 ← aresta anterior de v em P_i
        e^2 ← aresta posterior de v em P_i
        q ← ConsultaÚltimoPasso(v, i − 1, s, (P_1, ..., P_{i−1}), (T_1, ..., T_{i−1}), (S_1, ..., S_{i−1}))
        r^1 ← Reflexão(q, v, e^1)
        r^2 ← Reflexão(q, v, e^2)
        Associa (r^1, r^2) a v e adiciona à S_i
    retorna S_i
```

2.1.3 Localização de Pontos no Mapa de Último Passo

Uma vez determinados os conjuntos T_i e S_i, objetiva-se implementar o algoritmo `LocalizaPonto`, responsável por localizar um ponto p no mapa de último passo S_i. Na primeira abordagem proposta, implementa-se esse procedimento de forma direta, verificando inicialmente todas as regiões associadas a vértices, em seguida todas as regiões associadas a arestas e, caso o ponto não pertença a nenhuma dessas regiões, retornando a região de travessia de P_i. Para verificar se um ponto $p \in \mathbb{R}^2$ pertence a uma região associada a um vértice v de P_i com raios r^1 e r^2, determina-se primeiramente se o ângulo de abertura entre r^1 e r^2 é inferior ou superior a 180°. Se o ângulo for inferior a 180°, então p pertence à região associada a v se, e somente se, o produto vetorial entre r^1 e $(p - v)$ for positivo **e** o produto vetorial entre $(p - v)$ e r^2 for negativo. Caso contrário, p pertence à região associada a v se, e somente se, o produto vetorial entre r^1 e $(p - v)$ for positivo **ou** o produto vetorial entre $(p - v)$ e r^2 for negativo. Essas situações encontram-se ilustradas na Figura 8.

Analogamente, deseja-se verificar se um ponto p pertence a uma região associada a uma aresta e de P_i. Para tanto, consideram-se u e v como os vértices que definem a aresta e, em sentido anti-horário, e toma-se r^1 como o segundo raio associado ao vértice u e r^2 como o primeiro raio associado ao vértice v em S_i. Assim, p pertence à região associada a e se, e somente se, estiver no lado externo da aresta e e entre os raios r^1 e r^2, conforme ilustrado na Figura 9.

Uma vez estabelecidos os critérios para verificar se um ponto pertence a uma região associada a um vértice ou a uma aresta de P_i, pode-se implementar o algoritmo `LocalizaPonto` de forma direta, conforme apresentado no Algoritmo 5. Realizam-se dois laços: o primeiro verifica todas as regiões associadas a vértices de P_i pertencentes a T_i e o segundo verifica todas as regiões associadas a arestas de P_i pertencentes a T_i. Caso o ponto p não pertença a nenhuma dessas regiões, retorna-se a região de travessia de P_i. Observe-se que são verificados apenas vértices de P_i presentes em T_i, o que soluciona o problema de considerar raios associados a vértices ausentes em T_i e que, portanto,



(a) Ângulo de abertura inferior a 180°.

(b) Ângulo de abertura superior a 180°.

Figura 8: Verificação se um ponto p está no cone definido pelo vértice v e pelos raios r^1 e r^2.

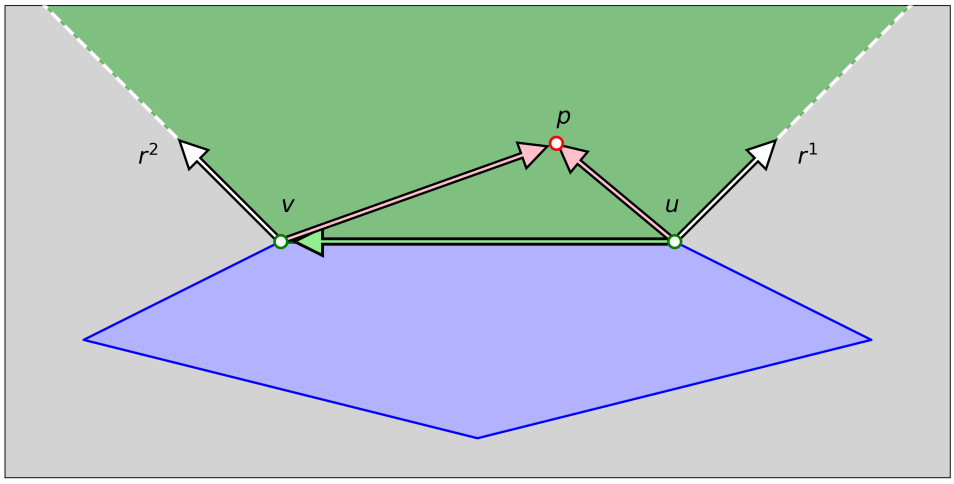


Figura 9: Verificação da pertinência de um ponto a uma região associada a uma aresta.

não definem regiões de vértice em S_i.

2.1.4 Implementação Completa

Uma vez estabelecidos os procedimentos para calcular T_i, S_i e localizar um ponto p em S_i, pode-se implementar o algoritmo completo para resolver o TPP, conforme apresentado no Algoritmo 6. Esse algoritmo itera sobre todos os polígonos $P_1, \ldots, P_k$, calculando T_i e S_i para cada polígono mediante os algoritmos previamente discutidos. Após calcular todas as regiões de primeiro contato e mapas de último passo, o algoritmo retorna o k-path até o ponto t utilizando o Algoritmo 1. Esse algoritmo também será empregado na segunda abordagem, na qual apenas o algoritmo `LocalizaPonto` será modificado.

Algoritmo 5: Localiza Ponto — Primeira Abordagem

Entrada: p, P_i, T_i, S_i
Saída : Região de S_i que contém p.

```
1  define LocalizaPonto(p, P_i, T_i, S_i):
2      para cada vértice v de P_i tal que existe uma aresta e em T_i incidente a v :
3          (r^1, r^2) ← raios associados a v em S_i
4          se r^1 × r^2 ≥ 0 :
5              se (r^1 × (p − v) > 0) ∧ (r^2 × (p − v) < 0) :
6                  retorna região associada a v
7          senão
8              se (r^1 × (p − v) > 0) ∨ (r^2 × (p − v) < 0) :
9                  retorna região associada a v
10     para cada aresta e = uv de P_i pertencente a T_i :
11         r^1 ← segundo raio associado a u em S_i
12         r^2 ← primeiro raio associado a v em S_i
13         se (r^1 × (p − u) > 0) ∧ (r^2 × (p − v) < 0) ∧ ((p − u) × (v − u) < 0) :
14             retorna região associada a e
15     retorna região de travessia de P_i
```

Algoritmo 6: TPP Irrestrito — Implementação Completa

Entrada: Pontos s e t, sequência de polígonos convexos disjuntos $(P_1, \ldots, P_k)$
Saída : k-path até t.

```
1  define TPP_Irrestrito(s, t, (P_1, ..., P_k)):
2      (T_1, ..., T_k) ← ∅
3      (S_1, ..., S_k) ← ∅
4      para cada i ← 1 até k :
5          T_i ← RegiãoDePrimeiroContato(i, s, (P_1, ..., P_i), (T_1, ..., T_{i−1}), (S_1, ..., S_{i−1}))
6          S_i ← MapaDeÚltimoPasso(i, s, (P_1, ..., P_i), (T_1, ..., T_i), (S_1, ..., S_{i−1}))
7      retorna ConsultaCaminho(t, k, s, (P_1, ..., P_k), (T_1, ..., T_k), (S_1, ..., S_k))
```

2.2 Segunda Abordagem

A implementação anterior é direta e adere rigorosamente à definição formal dos mapas de último passo. Todavia, sua eficiência é limitada, uma vez que o algoritmo `LocalizaPonto` possui complexidade $O(|T_i|)$ no pior caso, dado que pode ser necessário examinar todas as regiões associadas aos vértices e arestas de P_i para localizar um ponto p. Em virtude dessa limitação, propomos uma segunda abordagem para implementar o algoritmo `LocalizaPonto` de maneira mais eficiente, empregando uma estratégia de busca binária. Neste momento, manteremos a mesma implementação para os algoritmos de cálculo das regiões de primeiro contato e dos mapas de último passo; na terceira abordagem, discutiremos meios de otimizar esses algoritmos igualmente.

Inicialmente, propomos uma nova interpretação do mapa de último passo S_i de um polígono P_i e demonstramos que essa nova definição é equivalente à anterior. A nova definição é apresentada a seguir:

Nova definição de Mapa de Último Passo: O mapa de último passo S_i de um polígono P_i, $1 \leq i \leq k$, constitui uma partição do plano em regiões associadas a cada vértice de P_i e regiões associadas a cada aresta de P_i.

A região associada a um vértice v de P_i corresponde àquela definida anteriormente se $v \in T_i$; caso contrário, trata-se de uma semirreta que parte de v e possui a mesma direção do vetor $v - q$, onde q é o ponto que precede v no $(i-1)$-path até v.

A região associada a uma aresta $e = \overline{uv}$ de P_i é o conjunto de pontos situados no lado externo de e e entre o segundo raio associado a u e o primeiro raio associado a v em S_i. Os pontos na fronteira entre uma região de vértice e uma região de aresta pertencem à região de vértice. Os pontos na parte interna da aresta pertencem à região de aresta.

Finalmente, os pontos no interior de P_i são considerados pertencentes à região de atravessar P_i. Dessa forma, S_i define uma partição de $\mathbb{R}^2$.

A Figura 10 ilustra essa nova definição de mapa de último passo. Observa-se que as regiões associadas aos vértices e arestas presentes na região de primeiro contato coincidem com aquelas da definição anterior, ao passo que as regiões associadas aos vértices e arestas ausentes da região de primeiro contato particionam a região de atravessar P_i em regiões de vértices e de arestas, respectivamente.

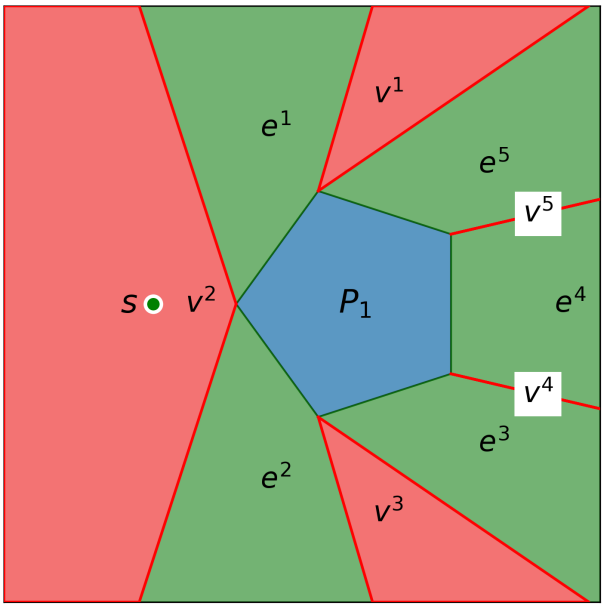


Figura 10: Nova interpretação do mapa de último passo S_1 de um polígono P_1. As regiões de vértice são representadas em vermelho e as de aresta em verde. Observe que, embora as arestas e^3, e^4, e^5 e os vértices v^4 e v^5 não estejam em T_1, possuem uma região associada.

Nota-se que as regiões são definidas de maneira idêntica para vértices e arestas na região de primeiro contato. Ademais, as regiões de vértice e de aresta que não pertencem à região de primeiro contato cobrem exatamente a região de atravessar P_i, desconsiderando o interior de P_i, o qual não é relevante, haja vista a hipótese de que os polígonos são disjuntos. Assim, para calcular o i-path até um ponto p, basta localizá-lo em S_i: se ele pertence à região de algum vértice ou aresta na região de primeiro contato, procede-se conforme anteriormente; caso contrário, conclui-se que p está na região de atravessar P_i e, portanto, o i-path até p é dado pelo $(i-1)$-path até p. Dessa forma, esta nova definição é equivalente à anterior.

A motivação para utilizar essa nova definição reside na garantia da seguinte propriedade, que é essencial para a implementação do algoritmo `LocalizaPonto` mediante uma estratégia de busca binária:

Arestas Fictícias: Seja P_i um polígono de m vértices e S_i seu mapa de último passo. Ademais, sejam v^l e v^r vértices de P_i, com $1 \leq l \leq r \leq m+1$, supondo que os vértices de P_i estão ordenados em sentido anti-horário e $v^{m+1} = v^1$. Dado $p \in \mathbb{R}^2$, desejamos determinar se existe algum $l \leq j < r-1$ tal que p pertence à região associada ao vértice v^j de P_i ou à região associada à aresta $\overline{v^j v^{j+1}}$ de P_i.

Para tal, poderíamos examinar cada região individualmente, porém essa estratégia não é eficiente. Entretanto, podemos criar uma *aresta fictícia* entre v^l e v^r e, utilizando o primeiro raio associado a v^l e o primeiro raio associado a v^r em S_i, definir uma região R que contém exatamente as regiões associadas às arestas entre v^l e v^r, aos vértices entre v^l e v^r e parte do interior de P_i. Demonstraremos a seguir que a região R é análoga à região de aresta tratada anteriormente. Ademais, é possível verificar se p pertence a R em tempo $O(1)$, e p pertence a R se e somente se pertence a alguma região de vértice ou de aresta entre v^l e v^r, visto que supomos que p não está no interior de P_i.

A Figura 11 ilustra essa ideia. Criamos uma aresta fictícia entre os vértices v^3 e $v^6 = v^1$ de P_1 e definimos a região R, que engloba todas as regiões entre eles. Assim, para verificar se existe algum $3 \leq j < 6$ tal que p pertence à região associada à aresta $\overline{v^j v^{j+1}}$ de P_1 ou à região associada ao vértice v^j de P_1, basta verificar se p pertence a R. Adicionalmente, R possui a forma de uma região de aresta tradicional, de modo que se pode verificar se p pertence a R empregando o mesmo procedimento do Algoritmo 5.

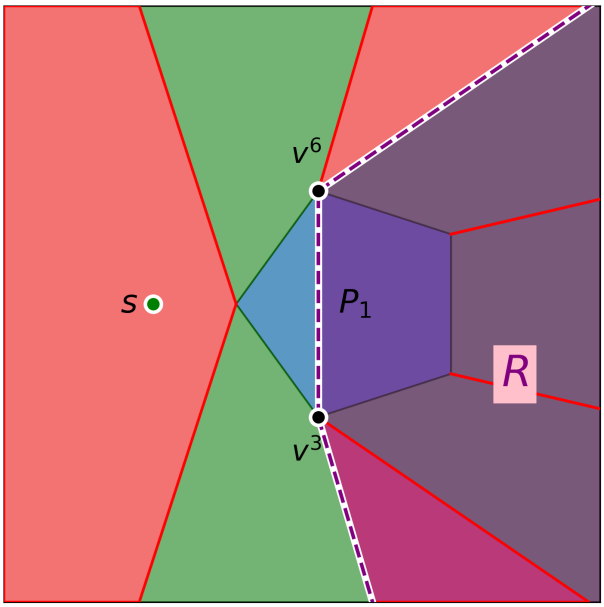


Figura 11: Criação de uma aresta fictícia entre os vértices v^3 e $v^6 = v^1$ de P_1, definindo a região R, que engloba todas as regiões dos vértices v^3, v^4 e v^5 e das arestas e^3, e^4 e e^5.

Antes de aplicar essa ideia propriamente, é necessário estabelecer um método para determinar se um ponto $p \in \mathbb{R}^2$ pertence à região R formada pelo vértice u com raio associado r^1 e pelo vértice v com raio associado r^2. Para tanto, define-se $d = v - u$ e estabelece-se que um raio r está do **lado externo** da aresta $\overline{uv}$ se $d \times r < 0$ e do **lado interno** se $d \times r \geq 0$. Além disso, calculam-se as seguintes variáveis booleanas:

1. $c_1 \iff r^1 \times (p - u) \geq 0$
2. $c_2 \iff r^2 \times (p - v) \leq 0$
3. $c_3 \iff (v - u) \times (p - u) \leq 0$

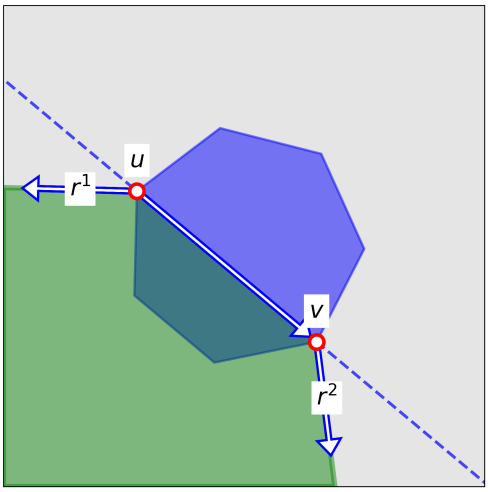
Observa-se que, considerando os quatro casos possíveis para os lados de r^1 e r^2, pode-se determinar se p pertence a R utilizando apenas c_1, c_2 e c_3:

(a) r^1 **e** r^2 **externos:** $c_1 \wedge c_2 \wedge c_3$. Veja a Figura 12(a).

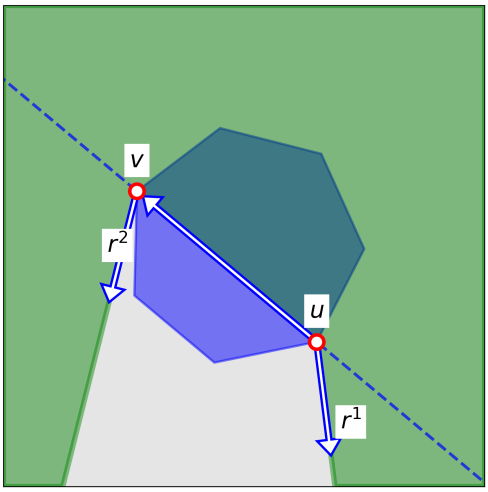
(b) r^1 **e** r^2 **internos:** $c_1 \vee c_2 \vee c_3$. Veja a Figura 12(b).

(c) r^1 **externo e** r^2 **interno:** $(c_1 \wedge c_3) \vee (c_2 \wedge \neg c_3)$. Veja a Figura 12(c).

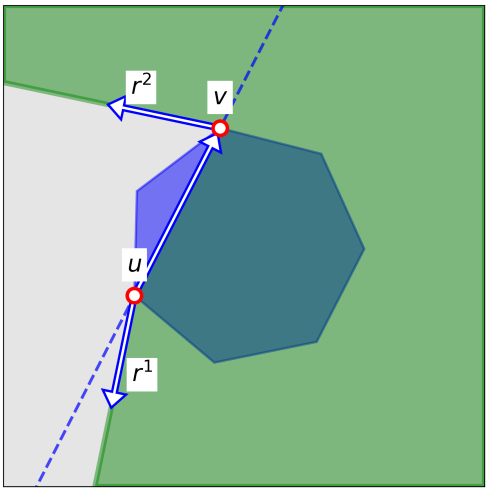
(d) r^1 **interno e** r^2 **externo:** $(c_1 \wedge \neg c_3) \vee (c_2 \wedge c_3)$. Veja a Figura 12(d).



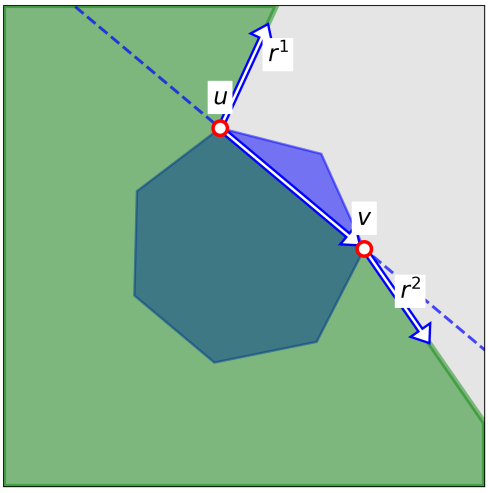
(a) r^1 e r^2 no lado externo.



(b) r^1 e r^2 no lado interno.



(c) r^1 no lado externo e r^2 no lado interno.



(d) r^1 no lado interno e r^2 no lado externo.

Figura 12: Ilustração dos quatro casos possíveis ao verificar se um ponto pertence à região de uma aresta fictícia entre dois vértices.

Uma vez estabelecido o método para verificar se um ponto $p \in \mathbb{R}^2$ pertence a uma região de aresta fictícia R, podemos utilizá-lo para localizar p no mapa de último passo S_i de um polígono P_i.

> **Algoritmo de localização:** Dado $1 \leq i \leq k$ e $p \in \mathbb{R}^2$ tal que p não pertence ao interior de P_i, deseja-se determinar a região de S_i que contém p. Definem-se $l_0 = 1$ e $r_0 = |P_i| + 1$, onde $|P_i|$ é o número de vértices de P_i. Para todo $i \in \mathbb{N}$, se $l_i + 1 \neq r_i$, define-se $mid_i = \lfloor (l_i + r_i)/2 \rfloor$

e verifica-se se p pertence à região delimitada pela aresta fictícia entre os vértices v^{l_i} e v^{mid_i} de P_i. Caso p pertença a essa região, definem-se $r_{i+1} = mid_i$ e $l_{i+1} = l_i$; caso contrário, definem-se $l_{i+1} = mid_i$ e $r_{i+1} = r_i$.

Eventualmente, alcança-se $l_i + 1 = r_i$ para algum i, momento no qual se conclui que p pertence à região associada ao vértice v^{l_i} ou à região associada à aresta $\overline{v^{l_i}v^{r_i}}$. Verifica-se, então, se $v^{l_i} \in T_i$ e se p pertence à região associada ao vértice v^{l_i}. Em caso afirmativo, retorna-se a região associada a v^{l_i}. Caso contrário, verifica-se se $\overline{v^{l_i}v^{r_i}} \in T_i$. Em caso afirmativo, retorna-se a região associada à aresta $\overline{v^{l_i}v^{r_i}}$. Caso contrário, retorna-se a região correspondente a atravessar P_i.

Esse algoritmo de localização encontra-se implementado no Algoritmo 7. Observa-se que, para implementar a função `VerificaAresta`, utiliza-se o procedimento descrito anteriormente para verificar se um ponto pertence à região delimitada por uma aresta fictícia entre dois vértices.

Considerando que implementamos o algoritmo `LocalizaPonto` mediante uma estratégia de busca binária, podemos substituir a implementação do Algoritmo 5 pela implementação do Algoritmo 7 no Algoritmo 6, mantendo inalterados os algoritmos de cálculo de regiões de primeiro contato e mapas de último passo.

A principal vantagem dessa abordagem reside no fato de que a busca binária reduz o número de verificações necessárias para localizar p em S_i de $O(|T_i|)$ para $O(\log|P_i|)$, o que pode representar uma melhoria significativa quando P_i possui muitos vértices em T_i. Analisaremos o impacto dessa modificação com maior profundidade na análise de complexidade.

2.3 Terceira Abordagem

Antes de apresentarmos a terceira abordagem, é necessário discutir a motivação para essa nova implementação do algoritmo `LocalizaPonto`. Considere o caso de um único polígono P_1 com um número elevado de vértices, por exemplo, um milhão. De acordo com a segunda abordagem, calculamos T_1 e, em seguida, S_1, para finalmente realizar uma única consulta ao mapa de último passo S_1 a fim de localizar o ponto t. Entretanto, essa consulta utiliza no máximo 20 regiões de vértice de P_1 para localizar t em S_1. Consequentemente, terminamos por calcular aproximadamente 999 980 regiões de vértice que nunca são efetivamente utilizadas. Diante dessa constatação, na terceira abordagem propomos empregar uma estratégia de memoização, de modo a calcular apenas as regiões de vértice de P_i que sejam efetivamente necessárias para localizar os pontos consultados no mapa de último passo S_i.

Essa abordagem reutilizará diversas das ideias apresentadas nas duas abordagens anteriores. Primeiramente, observe que os algoritmos de consulta (Algoritmo 1 e Algoritmo 2) não utilizam os valores de T_i e S_i explicitamente, limitando-se a repassá-los ao algoritmo `LocalizaPonto`. Assim, podemos manter esses dois algoritmos inalterados. Infelizmente, não é possível reutilizar diretamente os algoritmos que calculam as regiões de primeiro contato e os mapas de último passo (Algoritmo 3 e Algoritmo 4), uma vez que eles calculam T_i e S_i em sua totalidade, o que não é eficiente.

Retomando as conclusões de 2.1 e 2.2, se v um vértice de P_i com arestas incidentes e^1 e e^2, e seja q o último passo do $(i-1)$-path até v. Como v pertence tanto a e^1 quanto a e^2, temos que $e^1 \in T_i$ se e somente se q está localizado no lado externo de e^1, e $e^2 \in T_i$ se e somente se q está localizado no lado externo de e^2. Além disso, uma vez determinado se e^1 e e^2 pertencem a T_i, podemos calcular os raios associados a v em S_i utilizando o algoritmo de reflexão apresentado no Algoritmo 4.

A partir dessa observação, segue que é possível determinar se uma aresta e pertence a T_i utilizando apenas uma consulta ao $(i-1)$-path até um de seus vértices. Dessa forma, conseguimos calcular o cone de visibilidade de v em S_i e determinar se uma aresta e de P_i pertence a T_i mediante uma única consulta. É importante ressaltar que podemos armazenar os resultados dessas

Algoritmo 7: Localiza Ponto — Segunda Abordagem

Entrada: p, P_i, T_i, S_i
Saída : Região de S_i que contém p.

```
1  define VerificaAresta(u, v, S_i):
2      r^1 ← segundo raio associado a u em S_i
3      r^2 ← primeiro raio associado a v em S_i
4      c_1 ← r^1 × (p − u) ≥ 0
5      c_2 ← r^2 × (p − v) ≤ 0
6      c_3 ← (v − u) × (p − u) ≤ 0
7      d ← v − u
8      se (v − u) × r^1 < 0 :
9          se (v − u) × r^2 < 0 :
10             retorna c_1 ∧ c_2 ∧ c_3
11         senão
12             retorna (c_1 ∧ c_3) ∨ (c_2 ∧ ¬c_3)
13     senão
14         se (v − u) × r^2 < 0 :
15             retorna (c_1 ∧ ¬c_3) ∨ (c_2 ∧ c_3)
16         senão
17             retorna c_1 ∨ c_2 ∨ c_3
18 define LocalizaPonto(p, P_i, T_i, S_i):
19     v^1, ..., v^m ← vértices de P_i, ordenados em sentido anti-horário
20     l ← 1
21     r ← m + 1
22     while l + 1 ≠ r do
23         mid ← ⌊(l + r)/2⌋
24         se VerificaAresta(v^l, v^mid, S_i) :
25             r ← mid
26         senão
27             l ← mid
28     se v^l ∈ T_i e p está na região de vértice de v^l (Veja o Algoritmo 5) :
29         retorna região associada a v^l
30     senão se v^l v^r ∈ T_i :
31         retorna região associada a v^l v^r
32     senão
33         retorna região de atravessar P_i
```

consultas para evitar cálculos repetidos. Assim, calculamos os raios associados a um vértice de P_i e determinamos se uma aresta pertence a T_i uma única vez, retornando os resultados armazenados para consultas posteriores.

Dessa forma, podemos reutilizar a segunda implementação do algoritmo `LocalizaPonto` (Algoritmo 7). Para tanto, deixamos implícito que as linhas 2 e 3 do Algoritmo 7, que solicitam os raios associados aos vértices u e v em S_i, realizam as consultas necessárias para calcular os raios associados a u e v em S_i e armazenam os resultados para consultas futuras. Além disso, as linhas 28 e 30, que verificam se $v^l \in T_i$ e se $\overline{v^l v^r} \in T_i$, respectivamente, realizam as consultas necessárias

para determinar se v^l e $\overline{v^l v^r}$ pertencem a T_i e armazenam os resultados para consultas futuras.

Assim, concluímos a implementação da terceira abordagem do algoritmo `LocalizaPonto` e podemos finalizar a implementação completa do algoritmo TPP utilizando essa abordagem. Para isso, inicializamos $S_1, \ldots, S_k$ e $T_1, \ldots, T_k$ como conjuntos vazios e, em seguida, retornamos o k-path até t utilizando o Algoritmo 1. Observe que, nesse caso, o cálculo de T_i e S_i para cada $i = 1, \ldots, k$ é realizado de forma implícita durante as consultas ao mapa de último passo S_i, restringindo-se aos vértices e arestas que sejam efetivamente necessários para localizar os pontos consultados em S_i.

2.4 Análise de Complexidade

Analisaremos a seguir a complexidade de cada uma das três abordagens apresentadas para a resolução do TPP. Para tanto, adotaremos as seguintes notações: seja $|P_i|$ o número de vértices do polígono P_i e $|T_i|$ o número de arestas em T_i. Seja ainda k o número total de polígonos na sequência $P_1, \ldots, P_k$ e n a soma do número de vértices dos k polígonos.

2.4.1 Primeira Abordagem

Analisamos a complexidade da primeira abordagem, na qual o algoritmo `LocalizaPonto` é implementado de forma direta, verificando todas as regiões de vértice e de aresta associadas a T_i para localizar o ponto p.

LocalizaPonto 5: A complexidade deste algoritmo é $O(|T_i|)$, uma vez que, no pior caso, é necessário verificar todas as regiões de vértice e de aresta associadas a T_i para localizar o ponto p.

ConsultaCaminho 1 e ConsultaÚltimoPasso 2: Ambos os algoritmos iniciam com uma chamada ao algoritmo `LocalizaPonto`. O pior caso ocorre quando o ponto p pertence à região de travessia de P_i ou a uma região de aresta, o que requer uma nova consulta ao mapa de último passo S_{i-1}. Dessa forma, no pior caso, são realizadas i chamadas ao algoritmo `LocalizaPonto`, resultando em uma complexidade total de:

$$O\left(\sum_{j=1}^{i} |T_j|\right).$$

RegiãoDePrimeiroContato 3 e MapaDeÚltimoPasso 4: Ambos realizam uma única passagem pelos vértices de P_i, efetuando uma consulta para cada vértice. Consequentemente, a complexidade destes algoritmos é:

$$O\left(|P_i| \sum_{j=1}^{i} |T_j|\right).$$

TPP completo 6: O algoritmo executa uma chamada para calcular T_i e S_i para cada $i = 1, \ldots, k$ e, ao final, realiza uma chamada para consultar o caminho até o ponto t. Assim, a complexidade total do algoritmo TPP é dada por:

$$\begin{aligned} O\left(\sum_{i=1}^{k} |P_i| \sum_{j=1}^{i} |T_j| + \sum_{j=1}^{k} |T_j|\right) &= O\left(\sum_{i=1}^{k} |P_i| \sum_{j=1}^{i} |P_j| + \sum_{j=1}^{k} |P_j|\right) \\ &= O\left(\sum_{i=1}^{k} |P_i| \sum_{j=1}^{i} |P_j|\right) \end{aligned}$$

$$
\begin{aligned}
&= O\left(\sum_{i=1}^{k}|P_i|\sum_{j=1}^{k}|P_j|\right)\\
&= O\left(\sum_{i=1}^{k}|P_i|\cdot n\right)\\
&= O(n^2).
\end{aligned}
$$

Conclui-se, portanto, que a complexidade total do algoritmo TPP utilizando a primeira abordagem é $O(n^2)$.

2.4.2 Segunda Abordagem

Analisamos a seguir a complexidade da segunda abordagem, na qual o algoritmo `LocalizaPonto` é implementado por meio de uma estratégia de busca binária para localizar o ponto p no mapa de último passo S_i.

LocalizaPonto 7: A complexidade deste algoritmo é $O(\log|T_i|)$, correspondente ao pior caso da busca binária empregada para localizar o ponto p em S_i.

ConsultaÚltimoPasso 2 e ConsultaCaminho 1: De maneira análoga ao caso anterior, o pior caso envolve i chamadas ao algoritmo `LocalizaPonto`, resultando na complexidade:

$$O\left(\sum_{j=1}^{i}\log|T_j|\right).$$

RegiãoDePrimeiroContato 3 e MapaDeÚltimoPasso 4: Cada um destes algoritmos itera sobre os vértices de P_i e realiza uma consulta para cada vértice. Portanto, a complexidade é:

$$O\left(|P_i|\sum_{j=1}^{i}\log|T_j|\right).$$

TPP completo 6: O custo total do algoritmo é:

$$
\begin{aligned}
O\left(\sum_{i=1}^{k}|P_i|\sum_{j=1}^{i}\log|T_j| + \sum_{j=1}^{k}\log|T_j|\right) &= O\left(\sum_{i=1}^{k}|P_i|\sum_{j=1}^{i}\log|P_j|\right)\\
&= O\left(\sum_{i=1}^{k}|P_i|\sum_{j=1}^{k}\log|P_j|\right)\\
&= O\left(n\sum_{j=1}^{k}\log|P_j|\right).
\end{aligned}
$$

Observando que a soma $\sum_{j=1}^{k}\log|P_j|$ é maximizada quando todos os polígonos possuem o mesmo número de vértices, isto é, quando $|P_j| = n/k$ para todo j, obtém-se que a complexidade final do algoritmo TPP utilizando a segunda abordagem é:

$$O(nk\log(n/k)).$$

Conclui-se, assim, que a complexidade final do algoritmo TPP utilizando a segunda abordagem é $O(nk\log(n/k))$. Esta complexidade coincide exatamente com aquela apresentada por Dror et al. (2003) [2], o que indica que a implementação proposta atinge a mesma eficiência assintótica da solução original.

2.4.3 Terceira Abordagem

Resta, por fim, analisar a complexidade da terceira abordagem. Esta análise é mais delicada, uma vez que a complexidade do algoritmo `LocalizaPonto` passa a depender fortemente da instância específica, em particular da distribuição dos vértices nos polígonos e das consultas efetivamente realizadas ao mapa de último passo.

Não obstante, é imediato observar que a terceira abordagem não apresenta desempenho inferior ao da segunda, visto que, no pior caso, todas as regiões de vértice em T_i são calculadas, recuperando exatamente o comportamento da segunda abordagem. Assim, conclui-se que a complexidade da terceira abordagem é $O(nk \log(n/k))$ no pior caso, podendo ser significativamente inferior em instâncias favoráveis, a depender do padrão das consultas realizadas.

2.5 Experimentos

3 TPP Não Convexo

TODO: Implementar o algoritmo TPP para o caso de polígonos não convexos.

4 Conclusão

TODO: Concluir o texto, discutir possíveis melhorias e trabalhos futuros.

Referências

[1] E. M. Arkin, S. P. Fekete, and J. S. B. Mitchell. The traveling salesman problem with neighborhoods: A survey. In G. Gutin and A. P. Punnen, editors, *The Traveling Salesman Problem and Its Variations*, pages 377–443. Springer, Boston, MA, 2005.

[2] M. Dror, A. Efrat, A. Lubiw, and J. S. B. Mitchell. Touring a sequence of polygons. In *Proceedings of the Thirty-Fifth Annual ACM Symposium on Theory of Computing*, STOC '03, pages 473–482, New York, NY, USA, 2003.

[3] J. Liu and H. Liu. Research on path optimization method for warehouse inspection robot. *Applied Artificial Intelligence*, 37(1):2254048, 2023.

[4] K. J. Obermeyer. Path planning for a UAV performing reconnaissance of static ground targets in terrain. In *AIAA Guidance, Navigation, and Control Conference*, Chicago, IL, USA, 2009.